

DRIVELINK

Team Development Handbook

Group 026-Y2-S1-MLB-WEB1G2-03

A full-stack web application for issuing driving licenses online

React · Spring Boot · SQL Server

Table of Contents

Table of Contents	2
1. Project Overview	3
2. How Data Flows Right Now (Mock — What You See Today)	3
3. How Data Should Flow After Your Work (Real — The Goal)	3
4. What Already Exists (DO NOT MODIFY)	4
5. Your Assignment — Who Does What	4
6. The Development Recipe (Follow This Exactly)	5
Step A — Create Your Entity	5
Step B — Create Your Repository	5
Step C — Create Your Service + Controller	6
Step D — Connect to React	6
7. How to Test Your Work	6
Test Backend First (Postman)	6
Test Frontend Connection	7
8. How to Set Up Your Machine	7
Prerequisites	7
GitHub Repository	7
Setup Steps (First Time)	7
Starting Work on an Existing Clone (Every Day After)	7
9. GitHub Workflow (MANDATORY — No Exceptions)	8
Your Daily Workflow	8
Rules	9
10. Key Files to Study Before You Start Coding	9
11. Common Mistakes to Avoid	9

1. Project Overview

DriveLink is a full-stack web application for issuing driving licenses online.

Layer	Technology	Where it lives
Frontend (UI)	React + Vite + Tailwind	/frontend
Backend (API)	Java Spring Boot	/backend
Database	MS SQL Server (SSMS)	Your local machine

The frontend is already 100% built — all dashboard pages, forms, buttons, tables, and navigation for every single role. But right now it runs on fake data stored in the browser's `localStorage`. Your job is to connect it to the real Java backend so that data lives in SQL Server instead.

2. How Data Flows Right Now (Mock — What You See Today)

```

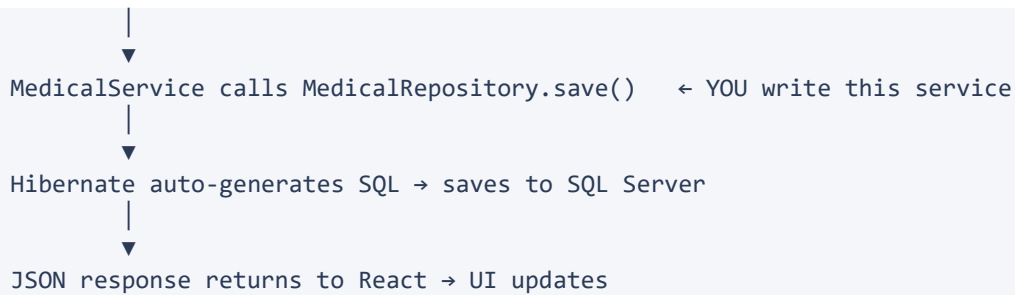
User clicks "Book Medical" on the UI
    ↓
React calls useStore().bookMedical(data)    ← store.tsx
    ↓
Data is saved to localStorage (browser)      ← FAKE, disappears if you clear cache
    ↓
UI re-renders showing the booking
  
```

The file `store.tsx` is the fake database. Every single page currently imports `useStore()` and calls functions like `bookMedical()`, `createApplication()`, `submitTrialResult()`, etc. These functions write to `localStorage` — not to a real database.

3. How Data Should Flow After Your Work (Real — The Goal)

```

User clicks "Book Medical" on the UI
    ↓
React calls bookMedical() from api.ts        ← YOU write this function
    ↓
fetch('/api/medical/book', { method: 'POST' }) ← Vite proxy forwards to :8080
    ↓
Spring Boot MedicalController receives it    ← YOU write this controller
  
```



The Vite proxy (already configured in vite.config.ts) silently forwards any `fetch('/api/...')` from the React app to `http://localhost:8080` where Spring Boot is running. You never type `localhost:8080` in React code.

4. What Already Exists (DO NOT MODIFY)

These files form the shared foundation. Do not edit them unless Karunaratne says so:

File	Purpose
backend/BackendApplication.java	Starts the server
backend/config/CorsConfig.java	Lets React talk to Java
backend/model/User.java	The users table
backend/model/Role.java	The role enum
backend/repository/UserRepository.java	User queries
backend/service/AuthService.java	Login & Register logic
backend/controller/AuthController.java	/api/auth/login and /api/auth/register
backend/dto/LoginRequest.java	Login request body
backend/dto/RegisterRequest.java	Register request body
backend/application.properties	DB connection config
frontend/vite.config.ts	Vite proxy to Spring Boot
frontend/src/lib/api.ts	The API bridge (your section is marked)
frontend/src/lib/auth.tsx	Session management
frontend/src/lib/store.tsx	The mock data layer (the "fake database")
frontend/src/types.ts	All TypeScript interfaces
frontend/src/components/	Everything here — shared UI components

5. Your Assignment — Who Does What

Each of you owns one major function. You will write 4 Java files in the backend, add your API functions in `api.ts`, and then update your assigned React pages to call those functions instead of `useStore()`.

#	Member	Module	Backend Endpoints	React Pages You Own
1	Weerasinghe (IT25100817)	Medical Test Booking	<code>/api/medical/*</code>	BookMedical.tsx, medical/Appointments.tsx, medical/Dashboard.tsx
2	Simra (IT25100815)	License Application	<code>/api/applications/*</code>	Apply.tsx, ApplicationDetail.tsx
3	Madampage (IT25100851)	Driving Exam Booking	<code>/api/exams/*</code>	BookExam.tsx, coordinator/Schedules.tsx, coordinator/Bookings.tsx
4	Fernando (IT25100821)	Driving Trial Booking	<code>/api/trials/*</code>	BookTrial.tsx, trainer/Trainees.tsx, trainer/Schedule.tsx
5	Thasalogithan (IT25100830)	License Approval	<code>/api/licenses/*</code>	officer/Applications.tsx, officer/Review.tsx, officer/Reports.tsx
6	Karunarathne (IT25100856)	License Renewal	<code>/api/renewals/*</code>	Renew.tsx (backend already done)

6. The Development Recipe (Follow This Exactly)

You have 4 steps per module. Here is what you do, in order.

Step A — Create Your Entity

Look at `frontend/src/types.ts`. Find the TypeScript interface that matches your module. For example, if you're doing Medical Booking, find `MedicalAppointment`. That interface tells you exactly what columns your SQL table needs.

Create a Java class in `backend/src/main/java/com/drivelink/backend/model/` that mirrors those fields using JPA annotations.

Hints

- Every entity needs `@Entity`, `@Table(name = "your_table")`, `@Id`, and `@GeneratedValue`
- Look at how `User.java` and `RenewalRequest.java` are written — follow the same pattern
- Every field needs a getter and a setter

Step B — Create Your Repository

Create an interface in `backend/.../repository/` that extends `JpaRepository`. This gives you `.save()`, `.findAll()`, `.findById()`, and `.deleteById()` for free.

If you need custom queries (e.g., "find all bookings for a specific user"), add method signatures like:

```
List<YourEntity> findByApplicantId(String applicantId);
```

Spring generates the SQL automatically from the method name. Look at how

Spring generates the SQL automatically from the method name. Look at how `RenewalRepository.java` does it.

Step C — Create Your Service + Controller

- Service = where your business logic goes (validation, status changes, etc.)
- Controller = the REST API endpoints that the React frontend will call

Your controller must provide at minimum these 4 endpoints (CRUD):

Method	Path	Action
POST	/api/your-module	Create
GET	/api/your-module	Read All
PUT	/api/your-module/{id}	Update
DELETE	/api/your-module/{id}	Delete

Look at `RenewalService.java` and `RenewalController.java` as your reference. Copy the structure, change the names.

Step D — Connect to React

Open `frontend/src/lib/api.ts`. Find your section (it's labeled with your name and IT number). Replace the `TODO` comments with real functions that call your backend using `apiFetch`.

Then open your assigned React page. Find where it calls `useStore().someFunction()` and replace that call with your new function from `api.ts`.

Example thought process for Weerasinghe

1. Open `BookMedical.tsx`
2. See it calls `useStore().bookMedical(data)` on line 11
3. That `bookMedical` is a mock function in `store.tsx` that writes to `localStorage`
4. You replace it with a call to your new `bookMedicalAPI(data)` function from `api.ts`
5. That function does `fetch('/api/medical/book', { method: 'POST', body: data })`
6. Which hits your `MedicalController.java` → `MedicalService.java` → SQL Server

7. How to Test Your Work

Test Backend First (Postman)

Before touching any React code, verify your Java API works:

1. Open Postman
2. Send a POST request to `http://localhost:8080/api/your-module` with a JSON body
3. Check that the response returns the saved object with an auto-generated id

4. Open SSMS → expand drivelink_db → Tables → verify the row exists

If this works, your backend is solid. Move on to Step D.

Test Frontend Connection

1. Make sure Spring Boot is running in IntelliJ (port 8080)
2. Make sure React is running via npm run dev in the /frontend folder (port 5173)
3. Open <http://localhost:5173> in your browser
4. Use the feature you connected (e.g., book a medical test)
5. Open SSMS and verify the new row appeared in your table

8. How to Set Up Your Machine

Every team member needs to do this once before starting development.

Prerequisites

- IntelliJ IDEA (Community or Ultimate) — used for BOTH backend and frontend
- SQL Server + SQL Server Management Studio (SSMS)
- Node.js (v18+)
- Git

GitHub Repository

```
https://github.com/Kithdhara/Drivelink
```

You have already been added as a contributor. Do not clone the repo from scratch — it already exists on your machine after cloning once.

Setup Steps (First Time)

1. Open IntelliJ IDEA
2. Get → VCS → Git → Clone → paste <https://github.com/Kithdhara/Drivelink> → choose a local folder
3. Open SSMS → create an empty database called drivelink_db
4. Open backend/src/main/resources/application.properties → change the spring.datasource.password to YOUR SQL Server sa password (do not commit this change)
5. Open the backend folder in IntelliJ → wait for Maven to download all dependencies (bottom progress bar)
6. Run BackendApplication.java (click ► green button)
7. Open IntelliJ's built-in terminal: View → Tool Windows → Terminal
8. In the terminal: cd frontend → npm install → npm run dev
9. Open <http://localhost:5173> in your browser

Starting Work on an Existing Clone (Every Day After)

```
git checkout main
git pull origin main          ← ALWAYS pull first to get latest code
git checkout feature/your-branch-name
git merge main                ← Bring in latest changes before starting
```

Both servers (IntelliJ Spring Boot + npm run dev) must be running simultaneously for the app to work.

IMPORTANT

Main branch is protected. You cannot push directly to main. You must create a Pull Request on GitHub and wait for Karunarathne's approval before your code is merged.

9. GitHub Workflow (MANDATORY — No Exceptions)

Nobody pushes directly to main. The main branch is protected — Karunarathne must review and approve every Pull Request before code is merged.

<https://github.com/Kithdhara/Drivelink>

```
main ← PROTECTED. No direct pushes. PR + Karunarathne approval required.
├── feature/weerasinghe-medical
├── feature/simra-applications
├── feature/madampage-exams
├── feature/fernando-trials
├── feature/thasalogithan-approval
└── feature/karunarathne-renewal
```

Your Daily Workflow

```
# 1. Start your day – pull latest code
git checkout main
git pull origin main

# 2. Switch to YOUR branch (create it the first time only)
git checkout -b feature/your-name-module # first time only
git checkout feature/your-name-module    # every day after

# 3. Bring in any new changes from main
git merge main

# 4. Write your code...

# 5. Push your progress
git add .
git commit -m "Short description of what you did"
git push origin feature/your-name-module
```

```
# 6. When your module is FULLY tested and working:
#   Go to https://github.com/Kithdhara/Drivelink
#   Click 'Compare & Pull Request' → assign Karunarathne as reviewer
#   Do NOT merge it yourself – wait for approval
```

Rules

- ONLY edit files inside your module (your entity, repository, service, controller, and your section of api.ts)
- Do NOT edit store.tsx, auth.tsx, types.ts, vite.config.ts, or any other member's files
- Do NOT edit application.properties — change your password locally but never commit that file
- Test in Postman before pushing. If your code doesn't compile, fix it before pushing

10. Key Files to Study Before You Start Coding

Read these files carefully. They are your reference implementation.

File	Why You Should Read It
types.ts	Shows you exactly what fields your entity needs
store.tsx	Shows you what mock functions exist (the ones you're replacing)
api.ts	Shows you where your section is and how apiFetch works
User.java	Shows you how to write a JPA Entity
RenewalRepository.java	Shows you how to write a Repository
RenewalService.java	Shows you how to write a Service
RenewalController.java	Shows you how to write REST endpoints
Your assigned React page	Shows you which useStore() calls you need to replace

11. Common Mistakes to Avoid

Mistake	What Happens	How to Avoid
Forgetting getters/setters in your Entity	Spring Boot can't read/write fields → 500 error	Add get/set for every field
Wrong @RequestMapping path	Frontend can't find your API → 404	Match the path in api.ts comments exactly
Not running both servers	React loads but API calls fail silently	Always have IntelliJ AND npm run dev running

Mistake	What Happens	How to Avoid
Editing someone else's files	Merge conflicts that are impossible to resolve	Stay in your lane
Pushing to main directly	You break everyone's working code	Use feature branches + Pull Requests
Not testing in Postman first	You spend hours debugging React when the Java API itself is broken	Always verify backend with Postman before touching frontend